

Yelra Yellow Paper & Formal Specification

Version 0.1 (Draft)

YELRA Project

Last update: October 25, 2025

Abstract

We present the draft specification of the Yelra programming language: design goals, core syntax, static semantics (typing), dynamic semantics (operational), memory model, and conformance requirements. This document is normative where explicitly stated; other sections are informative and provide rationale, examples, and security guidance.

Contents

1	Scope and Non-Goals	4
2	Design Principles	4
3	Lexical Structure (Normative)	4
3.1	Tokens	4
4	Surface Syntax (Informative)	5
5	Core Language (Normative)	5
5.1	Types	5
5.2	Terms	5
5.3	Typing (Static Semantics)	5
5.4	Dynamic Semantics (Operational)	6
5.5	Metatheory (Sketch)	6
6	Desugaring (Normative)	6
7	Type Declarations and ADTs (Normative)	7
8	Modules and Imports (Normative)	7
9	Memory Model (Normative)	7
10	Errors and Diagnostics (Informative)	7
11	Standard Library Kernel (Informative)	7
12	Security Model (Informative)	7
13	Performance Model (Informative)	7
14	Conformance (Normative)	8
14.1	Test Suite Structure	8
15	Versioning and Stability	8
16	Reference Implementation (Informative)	8
17	Implementation Pipeline (Informative)	8
17.1	lexer.rs – the cutter	8
17.2	parser.rs – the organizer	9
17.3	ast.rs – the boxes/blueprints	9

17.4 <code>eval.rs</code> – the cook	9
18 Parser Walkthrough (Informative)	9
18.1 Imports (we grab our Lego pieces)	9
18.2 The Parser backpack (what it carries)	10
18.3 New parser (clean up and start at the front)	13
18.4 Peek and Next (look vs. move)	14
18.5 <code>parse_expr</code> (turn cards into a little meaning-tree)	14
18.6 Infix sugar: turn <code>1 + 2</code> into <code>(+ 1 2)</code>	18
18.7 Top-level parse function (one expression only, no leftovers)	20
19 Worked Mini-Example (Informative)	21
20 Glossary	21

1 Scope and Non-Goals

This yellow paper is intended to be implementable with minimal ambiguity. We target:

- A small, well-specified core calculus (§5) with total, deterministic evaluation.
- A practical surface language (§4) that desugars to the core.
- A sound type system with principal types for the core, and *checked effects* as an extension.

Non-goals for v0.1: concurrency semantics, FFI beyond the **extern** stub, and formal cost semantics.

Remarks 1.1. Use this section to log exclusions so implementers do not over-interpret the v0.1 guarantees.

2 Design Principles

1. **Minimal core, rich edges:** keep the core small (functions, products, sums, recursion), push features to libraries and desugaring.
2. **Deterministic evaluation:** the same program yields the same result on any conforming implementation.
3. **Static clarity first:** errors should be rejected at compile-time where feasible, with precise spans and messages.
4. **Ergonomics by desugaring:** sugar like `pattern match` and pipeline operators lower into the core.

3 Lexical Structure (Normative)

Tokens are defined by a DFA over Unicode scalar values; source files are UTF-8.

3.1 Tokens

The token set includes identifiers, keywords, literals, operators, delimiters, and layout. Whitespace and line comments are skipped by the lexer.

3.1.0.1 Identifiers

$$\langle Ident \rangle ::= _A-Za-z _A-Za-z0-9^*$$

Identifiers beginning with an underscore are permitted and have no special semantics.

3.1.0.2 Literals

$$\langle IntLit \rangle ::= 0 \mid [1-9][0-9]^* \quad \langle BoolLit \rangle ::= \text{true} \mid \text{false} \quad \langle StrLit \rangle ::= "(\backslash n \mid \dots)"$$

3.1.0.3 Operators

The following operator lexemes are reserved and left-associative unless stated: `+` `-` `*` `/` `==` `!=` `<` `<=` `>` `>=` `->` `::` `|` `&` `&&` `||` `=>`.

4 Surface Syntax (Informative)

We define the programmer-facing constructs; each form desugars to the core language (§5). Examples:

```
1 // simple bindings
2 let x = 41
3 let y = x + 1
4
5 // functions
6 fn add(a: Int, b: Int): Int { a + b }
7
8 // algebraic data types (sugar over sums/products)
9 type Option[T] = Some(T) | None
10
11 // pattern matching (desugars to case)
12 match opt with
13   | Some(v) => v
14   | None    => 0
```

5 Core Language (Normative)

5.1 Types

$$\begin{aligned} \tau ::= & \text{Unit} \mid \text{Int} \mid \text{Bool} \mid \text{Str} \mid \tau \rightarrow \tau \\ & \mid \tau \times \tau \mid \tau + \tau \mid \text{List } \tau \mid \text{Rec}\{\overline{\ell : \tau}\} \end{aligned}$$

5.2 Terms

$$\begin{aligned} e ::= & x \mid () \mid n \mid b \mid s \mid \lambda x : \tau. e \mid ee \mid \text{fix } f. \lambda x : \tau. e \\ & \mid (e, e) \mid \text{fst } e \mid \text{snd } e \\ & \mid \text{inl } e \mid \text{inr } e \mid \text{case } e \text{ of } \text{inl } x \Rightarrow e \mid \text{inr } y \Rightarrow e \\ & \mid \{\overline{\ell = e}\} \mid e.\ell \mid \text{let } x = e \text{ in } e \end{aligned}$$

5.3 Typing (Static Semantics)

Typing judgments have the form $\Gamma \vdash e : \tau$, where Γ is a finite map from variables to types.

5.3.0.1 Selected rules

$$\begin{array}{c}
\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{T-VAR} \quad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x : \tau_1. e : \tau_1 \rightarrow \tau_2} \text{T-ABS} \\
\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 e_2 : \tau_2} \text{T-APP} \quad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{T-LET}
\end{array}$$

Remarks 5.1. We aim for principal types in the core; surface-level inference is by elaboration then checking.

5.4 Dynamic Semantics (Operational)

We specify a small-step semantics $e \rightarrow e'$ under a call-by-value strategy.

5.4.0.1 Values

$$v ::= () \mid n \mid b \mid s \mid \lambda x : \tau. e \mid (v, v) \mid \text{inl } v \mid \text{inr } v \mid \{\overline{\ell = v}\}$$

5.4.0.2 Evaluation contexts (informal)

Evaluation proceeds left-to-right in applications, pairs, records, and case scrutinees.

5.4.0.3 Selected rules

$$\begin{array}{c}
\frac{}{(\lambda x : \tau. e) v \rightarrow e[x \mapsto v]} \text{E-APP1} \quad \frac{}{\text{let } x = v \text{ in } e \rightarrow e[x \mapsto v]} \text{E-LET} \\
\frac{}{\text{case inl } v \text{ of inl } x \Rightarrow e_1 \mid \text{inr } y \Rightarrow e_2 \rightarrow e_1[x \mapsto v]} \text{E-CASEL} \\
\frac{}{\text{fst } (v_1, v_2) \rightarrow v_1} \text{E-FST} \quad \frac{}{\text{snd } (v_1, v_2) \rightarrow v_2} \text{E-SND}
\end{array}$$

5.5 Metatheory (Sketch)

Theorem 5.2 (Progress). *If $\emptyset \vdash e : \tau$, then either e is a value or there exists e' such that $e \rightarrow e'$.*

Theorem 5.3 (Preservation). *If $\Gamma \vdash e : \tau$ and $e \rightarrow e'$, then $\Gamma \vdash e' : \tau$.*

6 Desugaring (Normative)

Each surface construct lowers to core terms. For example, `match` on a binary sum:

$$\begin{array}{c}
\text{match } e \text{ with } \mid C_1(x) \Rightarrow e_1 \mid C_2(y) \Rightarrow e_2 \\
\Downarrow \\
\text{case } e \text{ of inl } x \Rightarrow e_1 \mid \text{inr } y \Rightarrow e_2
\end{array}$$

7 Type Declarations and ADTs (Normative)

Algebraic data types expand to sums/products. Kind checking for parametrized types ensures well-formedness.

7.0.0.1 Example

$$\text{Option } \alpha \triangleq \alpha + \text{Unit} \quad \text{Some}(v) \triangleq \text{inl } v \quad \text{None} \triangleq \text{inr } ()$$

8 Modules and Imports (Normative)

A module is a map from exported names to definitions with an explicit interface. Import resolution is deterministic and acyclic for v0.1.

9 Memory Model (Normative)

The core language is pure; mutation is provided via an effect **State** as a library extension with monadic typing discipline. Reference identity and aliasing are not observable in the pure core.

Remarks 9.1. For the Rust reference implementation, we commit to *value semantics* at the core boundary and isolate interior mutability behind well-typed effects.

10 Errors and Diagnostics (Informative)

Compile-time errors must report a span, category, and suggestion. Runtime errors are not permitted in well-typed core programs.

11 Standard Library Kernel (Informative)

The kernel provides `Int`, `Bool`, `Str`, `List`, `Option`, `Result`, records, and basic I/O via an effect boundary.

12 Security Model (Informative)

- **Determinism:** eliminates a class of timing channels in the core.
- **Totality:** no unchecked exceptions in the core calculus.
- **FFI barrier:** `extern` functions must declare effects explicitly; the type checker enforces containment.

13 Performance Model (Informative)

We specify asymptotic costs for core constructs and leave constant factors to the implementation. Tail-call optimization for `fix`-encoded recursion is recommended.

14 Conformance (Normative)

An implementation is *core-conforming* if:

- (C1) It accepts precisely the well-typed core programs and rejects ill-typed ones (§5).
- (C2) It evaluates according to the small-step semantics and yields identical results for terminating programs.
- (C3) It implements the desugaring map for the surface subset it claims to support (§6).

14.1 Test Suite Structure

Conformance packs are organized as:

- **positive/**: well-typed programs with golden outputs.
- **negative/**: programs that must be rejected with specific diagnostic categories.

15 Versioning and Stability

Language versions follow MAJOR.MINOR. Minor changes may add syntax sugar that desugars to existing core constructs. Any change to the core types or semantics increments MAJOR.

16 Reference Implementation (Informative)

The reference interpreter is written in Rust. The lexer (logos), parser (nom/lalrpop), AST, and evaluator map directly to the formal sections:

- **lexer.rs** → §3
- **parser.rs, ast.rs** → §4, §5
- **eval.rs** → §5 (operational semantics)

17 Implementation Pipeline (Informative)

Think of the language like a little bakery that turns a recipe ($1 + 1$) into a cake (2). Each file is a helper with a simple job.

17.1 **lexer.rs** – the cutter

Purpose: chop the raw text into small pieces called *tokens*.

Rationale: computers need neat pieces, not one long string.

Meaning: it reads " $1 + 1$ " and cuts it into ["1", "+", "1"] with labels like `Number`, `Symbol`, `LParen`, `RParen`, etc.

Kid version: scissors that cut sentences into word-cards.

17.2 parser.rs – the organizer

Purpose: take the token cards and build a *structure* (a little tree) that shows how things fit together.

Rationale: $1 + 1$ isn't just three cards; it's *add* with two numbers.

Meaning: turns tokens into an *AST* shape like `List([Symbol("+"), Number(1), Number(1)])`.

Kid version: arrange the word-cards into a mini Lego model that says “this is an add”.

17.3 ast.rs – the boxes/blueprints

Purpose: define the *shapes* of those Lego models (the tree nodes).

Rationale: everyone needs to agree on what a “Number” or “Add list” looks like in code.

Meaning: declares types like `Expr::Number`, `Expr::Symbol`, `Expr::List` so the parser and evaluator talk the same language.

Kid version: the picture guide that says “this block is a Number, this block is a Symbol”.

17.4 eval.rs – the cook

Purpose: read the AST and *do the thing* (compute a result).

Rationale: after we've built the “add” model, someone must actually add the numbers.

Meaning: walks the AST; when it sees $(+ \ 1 \ 1)$, it returns 2.

Kid version: the baker who follows the model/recipe and makes the cake.

How they work together (quick story)

1. You type: $1 + 1$
2. `lexer.rs` cuts it $\rightarrow$ tokens: `Number("1"), Symbol("+"), Number("1")`
3. `parser.rs` organizes it $\rightarrow$ AST: `List([Symbol("+"), Number(1), Number(1)])`
4. `eval.rs` cooks it $\rightarrow$ result: 2
5. `ast.rs` is the shared blueprint they all use to describe the pieces.

That's the whole pipeline: *cut* $\rightarrow$ *organize* $\rightarrow$ *blueprint-shaped tree* $\rightarrow$ *cook*.

18 Parser Walkthrough (Informative)

Let's read this file like a picture book. Yelra is learning to **read math sentences**. This file teaches it how.

18.1 Imports (we grab our Lego pieces)

```
1 use crate::ast::Expr;
2 use crate::lexer::Token;
```

“Bring the `Expr` shapes (our Lego models) and the `Token` cards (cut-up words) so we can use them.”

C# equivalent

```
1 // C# namespaces (example)
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5
6 // We'll define Token and Expr types below in C# snippets.
```

18.2 The Parser backpack (what it carries)

```
1 pub struct Parser {
2     tokens: Vec<Token>,
3     pos: usize,
4 }
```

The `Parser` holds: (i) `tokens`: the list of word-cards to read; (ii) `pos`: a finger pointing at which card we're on.

C# equivalent

```
1 // Minimal algebraic-style modeling for tokens and expressions
2 public abstract record Token;
3 public sealed record LParenToken() : Token;
4 public sealed record RParenToken() : Token;
5 public sealed record WhitespaceToken() : Token;
6 public sealed record NumberToken(string Text) : Token;
7 public sealed record SymbolToken(string Text) : Token;
8
9 public abstract record Expr;
10 public sealed record Number(double Value) : Expr;
11 public sealed record Symbol(string Name) : Expr;
12 public sealed record ListExpr(List<Expr> Items) : Expr;
13
14 public sealed class Parser
15 {
16     private readonly List<Token> tokens;
17     private int pos;
18
19     public Parser(IEnumerable<Token> tokens)
20     {
21         // whitespace is removed here (same as Rust new())
22         this.tokens = tokens.Where(t => t is not WhitespaceToken).ToList();
23         this.pos = 0;
24     }
```

```
25 // methods follow...
26 }
```

Term-by-term explanation (Rust meaning, plain language, intuitively)

Here is a plain-English tour of every symbol, keyword, and bit of jargon in:

```
pub struct Parser { tokens: Vec<Token>, pos: usize, }
```

pub

Rust meaning: visibility modifier (public outside the current module).

plain language: anyone in our project can use this.

intuitively: a classroom project we are allowed to show to the whole school, not just our table.

struct

Rust meaning: defines a composite data type with named fields (a record).

plain language: a custom box with labeled compartments.

intuitively: a lunchbox with sections labeled “sandwich” and “snack.”

Parser

Rust meaning: the name (identifier) of this struct type.

plain language: the model name of the box we designed.

intuitively: calling our lunchbox model “CrunchMaster 3000.”

{ ... } (after Parser)

Rust meaning: the body of the struct—lists its fields.

plain language: the blueprint that lists the compartments.

intuitively: a diagram showing which lunchbox sections exist.

tokens

Rust meaning: field name; holds a collection of `Token`s.

plain language: the pile of word-cards the parser will read.

intuitively: a stack of flashcards we go through one by one.

:

Rust meaning: type ascription operator; “has type.”

plain language: this thing is of kind...

intuitively: a label under a cubby that says “snacks go here.”

Vec<Token>

Rust meaning: growable vector (dynamic array) of elements of type **Token**; angle brackets indicate a generic.

plain language: a resizable list of tokens.

intuitively: a stretchy accordion folder holding lots of identical flashcards.

Vec

Rust meaning: standard library dynamic array type.

plain language: a resizable array container.

intuitively: a backpack that magically expands when we add more cards.

<Token> (generic parameter)

Rust meaning: specifies the element type for the vector (generics).

plain language: this is a list of Tokens (not apples, not screws).

intuitively: a Lego bin labeled “only blue 2-stud bricks.”

Token

Rust meaning: a type for lexical units (numbers, symbols, parentheses, etc.).

plain language: one chopped piece of the original text.

intuitively: a single flashcard with a symbol or number on it.

, (comma)

Rust meaning: separates fields; trailing commas are allowed.

plain language: next field coming up.

intuitively: a divider between items in a packing list.

pos

Rust meaning: field name; tracks the current index into `tokens`.

plain language: the cursor/finger pointing at where we are in the list.

intuitively: our finger marking which flashcard we are reading.

usize

Rust meaning: unsigned integer sized to the platform’s pointer width; ideal for indexing.

plain language: a whole number sized for our machine, used for positions/lengths.

intuitively: a page number big enough for any book our shelf can hold.

} (closing brace), no semicolon

Rust meaning: ends the struct body; no semicolon after a type definition.

plain language: blueprint complete.

intuitively: closing the lid on the lunchbox plan.

Mini big-picture

- The **struct** defines a **Parser** that owns two things:
 - `tokens`: a *resizable list* of `Token` pieces to read.
 - `pos`: a *position counter* (a machine-sized, non-negative integer) showing where we are looking.
- **Public** (`pub`) means other modules/crates can create and use this parser type.
- **Generics** (`Vec<Token>`) means “a container that specifically holds `Tokens`.”

If we keep the lunchbox image in mind: we built a public model called **Parser** with two compartments—one stretchy compartment for all the flashcards (`tokens`) and one little window showing the current card number (`pos`).

18.3 New parser (clean up and start at the front)

```
1 impl Parser {  
2     pub fn new(tokens: Vec<Token>) -> Self {  
3         let tokens: Vec<Token> = tokens  
4             .into_iter()  
5             .filter(|t| !matches!(t, Token::Whitespace))  
6             .collect();  
7  
8         Parser { tokens, pos: 0 }  
9     }
```

When we make a new parser: we throw away whitespace cards (spaces/tabs/newlines) so we don't trip over them, and set our finger (`pos`) to the start.

C# equivalent

```
1 // Already shown in the C# constructor above; same behavior:
2 // - filters out WhitespaceToken
3 // - sets pos = 0
```

18.4 Peek and Next (look vs. move)

```
1 fn peek(&self) -> Option<Token> {
2     self.tokens.get(self.pos)
3 }
```

`peek` = “Look at the current card, but don't move your finger.”

```
1 fn next(&mut self) -> Option<Token> {
2     if self.pos < self.tokens.len() {
3         let t = self.tokens[self.pos].clone();
4         self.pos += 1;
5         Some(t)
6     } else {
7         None
8     }
9 }
```

`next` = “Give me the current card and move my finger to the next card.” If we're past the last card, we get `None`.

C# equivalent

```
1 private Token? Peek() => pos < tokens.Count ? tokens[pos] : null;
2
3 private Token? Next()
4 {
5     if (pos < tokens.Count) return tokens[pos++];
6     return null;
7 }
```

18.5 parse_expr (turn cards into a little meaning-tree)

```
1 pub fn parse_expr(&mut self) -> Result<Expr, String> {
2     // Parse a primary expression first
3     let first = match self.next() {
```

We're going to read one expression. First, read a *primary* piece (a number, a symbol, or a parenthesized list).

18.5.0.1 Case: Number

```
1      Some(Token::Number(s)) => {
2          let n: f64 = s
3              .parse()
4              .map_err(|e| format!("Invalid number '{}': {}", s, e))?;
5          Expr::Number(n)
6      }
```

If the card says `Number("123")`, turn it into `Expr::Number(123.0)`; otherwise complain.

18.5.0.2 Case: bare Symbol

```
1      Some(Token::Symbol(s)) => {
2          // A bare symbol (not inside parentheses) -- return as symbol
3          Expr::Symbol(s)
4      }
```

If the card says `Symbol("+")` or `Symbol("x")`, make a `Expr::Symbol("...")`. We don't know usage yet.

18.5.0.3 Case: List starting with '('

```
1      Some(Token::LParen) => {
2          // parse list until matching RParen
3          let mut exprs = Vec::new();
4          while let Some(tok) = self.peek() {
5              if matches!(tok, Token::RParen) {
6                  // consume RParen
7                  self.next();
8                  return Ok(Expr::List(exprs));
9              } else {
10                 let e = self.parse_expr()?;
11                 exprs.push(e);
12             }
13         }
14         return Err("Unclosed '(' -- reached end of input".to_string());
15     }
```

Seeing (starts a list: read expressions until). Inside, we recursively call `parse_expr()` for each item. If we never find), error.

18.5.0.4 Case: Unexpected ')', whitespace, or end

```
1      Some(Token::RParen) => return Err("Unexpected ')'.to_string()),
```

```

2         Some(Token::Whitespace) => return Err("Unexpected whitespace token".to_string
3         ()),
4         None => return Err("Unexpected end of input".to_string()),
        };

```

A lone `)` is bad; whitespace shouldn't appear (we filtered it); and empty input is an error.

C# equivalent

```

1 public bool TryParseExpr(out Expr expr, out string? error)
2 {
3     // helper to emit errors
4     Expr? PrimaryFromToken(Token? tok, out string? err)
5     {
6         err = null;
7         if (tok is null) { err = "Unexpected end of input"; return null; }
8
9         switch (tok)
10        {
11            case NumberToken nt:
12                if (!double.TryParse(nt.Text, out var val))
13                {
14                    err = $"Invalid number '{nt.Text}'";
15                    return null;
16                }
17                return new Number(val);
18
19            case SymbolToken st:
20                return new Symbol(st.Text);
21
22            case LParenToken:
23                {
24                    var items = new List<Expr>();
25                    while (true)
26                    {
27                        var look = Peek();
28                        if (look is RParenToken)
29                        {
30                            Next(); // consume ')'
31                            return new ListExpr(items);
32                        }
33                        var ok = TryParseExpr(out var inner, out var innerErr);
34                        if (!ok) { err = innerErr; return null; }
35                        items.Add(inner);
36
37                        if (Peek() is null)

```

```

38         {
39             err = "Unclosed '(' -- reached end of input";
40             return null;
41         }
42     }
43 }
44
45 case RParenToken:
46     err = "Unexpected ')'";
47     return null;
48
49 case WhitespaceToken:
50     err = "Unexpected whitespace token";
51     return null;
52
53 default:
54     err = "Unknown token";
55     return null;
56 }
57 }
58
59 // 1) parse primary
60 var firstToken = Next();
61 var first = PrimaryFromToken(firstToken, out error);
62 if (first is null) { expr = null!; return false; }
63
64 // 2) infix sugar: only if first is Number or ListExpr
65 if (first is Number || first is ListExpr)
66 {
67     var operands = new List<Expr> { first };
68     var ops = new List<string>();
69
70     // collect (op, rhs)* while next token is a Symbol
71     while (Peek() is SymbolToken)
72     {
73         var opTok = Next() as SymbolToken; // consume operator
74         var ok = TryParseExpr(out var rhs, out var rhsErr);
75         if (!ok) { expr = null!; error = rhsErr; return false; }
76
77         ops.Add(opTok!.Text);
78         operands.Add(rhs);
79     }
80
81     if (ops.Count == 0)

```

```

82     {
83         expr = operands[0]; error = null; return true;
84     }
85     else if (ops.Count == 1)
86     {
87         var list = new List<Expr>();
88         list.Add(new Symbol(ops[0]));
89         list.AddRange(operands);
90         expr = new ListExpr(list); error = null; return true;
91     }
92     else
93     {
94         var same = ops.All(o => o == ops[0]);
95         if (!same)
96         {
97             expr = null!; error = "Mixed operators without parentheses are not
supported -- use parentheses to disambiguate.";
98             return false;
99         }
100        var list = new List<Expr>();
101        list.Add(new Symbol(ops[0]));
102        list.AddRange(operands);
103        expr = new ListExpr(list); error = null; return true;
104    }
105 }
106
107 // first was a Symbol: return as-is (no infix folding)
108 expr = first; error = null; return true;
109 }

```

18.6 Infix sugar: turn 1 + 2 into (+ 1 2)

```

1      // If the primary expression is a Number or a List, try to parse infix
continuation:
2      // pattern: first (Symbol op, Expr rhs)+
3      match &first {
4          Expr::Number(_) | Expr::List(_) => {
5              let mut operands: Vec<Expr> = vec![first.clone()];
6              let mut ops: Vec<String> = Vec::new();
7
8              // collect (op, rhs) pairs greedily, but only when the next token is a
Symbol
9              while let Some(Token::Symbol(_)) = self.peek() {
10                 // consume operator symbol

```

```

11         let op = match self.next() {
12             Some(Token::Symbol(s)) => s,
13             _ => unreachable!(),
14         };
15         // parse rhs expression
16         let rhs = self.parse_expr()?;
17         ops.push(op);
18         operands.push(rhs);
19     }

```

```

1     if ops.is_empty() {
2         // no infix continuation; just return the primary expr
3         Ok(operands.into_iter().next().unwrap())

```

```

1     } else if ops.len() == 1 {
2         // single operator: (op first rhs)
3         let op = ops[0].clone();
4         let mut list: Vec<Expr> = Vec::new();
5         list.push(Expr::Symbol(op));
6         list.extend(operands);
7         Ok(Expr::List(list))

```

```

1     } else {
2         // multiple operators: ensure they are all the same (left-associ, same
- op only)
3         let all_same = ops.iter().all(|o| o == &ops[0]);
4         if all_same {
5             let op0 = ops[0].clone();
6             let mut list: Vec<Expr> = Vec::new();
7             list.push(Expr::Symbol(op0));
8             list.extend(operands);
9             Ok(Expr::List(list))
10        } else {
11            Err("Mixed operators without parentheses are not supported -- use
parentheses to disambiguate.".to_string())
12        }
13    }

```

```

1    }
2    // If the first expression is a Symbol (e.g. user typed "+ 1 2" without
parentheses),
3    // don't attempt to treat following tokens as infix; just return the symbol
expression.
4    Expr::Symbol(_) => Ok(first),

```

```

5         }
6     }

```

If we saw many operators (like `1 + 2 + 3`), we only allow it if they're all the same (e.g., `+`), producing `(+ 1 2 3)`. Mixed operators (e.g., `+` and `*`) are rejected—use parentheses.

C# equivalent

```

1 // Covered inside TryParseExpr above.
2 // The same "all operators must match" rule is enforced with LINQ .All(...).

```

18.7 Top-level parse function (one expression only, no leftovers)

```

1 // top-level parse entry
2 pub fn parse(tokens: Vec<Token>) -> Result<Expr, String> {
3     let mut p = Parser::new(tokens);
4     let expr = p.parse_expr()?;

```

```

1     // Ensure we've consumed all tokens; if anything remains, report where we stopped.
2     if let Some(remaining) = p.peek() {
3         // Give a clearer message for debugging leftover tokens
4         let kind = match remaining {
5             Token::LParen => "'('".to_string(),
6             Token::RParen => "')'".to_string(),
7             Token::Number(n) => format!("number '{}'", n),
8             Token::Symbol(s) => format!("symbol '{}'", s),
9             Token::Whitespace => "whitespace".to_string(),
10        };
11        return Err(format!(
12            "Extra tokens after first expression (next token: {})",
13            kind
14        ));
15    }

```

```

1     Ok(expr)
2 }

```

We parse one expression into a tree and then ensure nothing remains; if there are leftovers, we report what's next for easier debugging.

C# equivalent

```

1 public static Expr ParseOne(IEnumerable<Token> tokens)
2 {
3     var p = new Parser(tokens);
4     if (!p.TryParseExpr(out var expr, out var err))

```

```

5      throw new InvalidOperationException(err);
6
7      var remaining = p.Peek();
8      if (remaining is not null)
9      {
10         var kind = remaining switch
11         {
12             LParenToken      => "('( ')",
13             RParenToken      => "')'",
14             NumberToken n    => $"number '{n.Text}' ",
15             SymbolToken s    => $"symbol '{s.Text}' ",
16             WhitespaceToken => "whitespace",
17             -                 => "unknown"
18         };
19         throw new InvalidOperationException(
20             $"Extra tokens after first expression (next token: {kind})");
21     }
22     return expr;
23 }

```

Summary

- We throw away spaces first.
- We read a number, a symbol, or a (...) list.
- We allow 1 + 2 (+ 3 ...) by turning it into the list (+ 1 2 3) when all ops match.
- Mixed ops like + and * together are rejected—use parentheses.
- We parse exactly one expression and make sure nothing is left behind.

19 Worked Mini-Example (Informative)

```

1 // factorial via fix
2 fn fact(n: Int): Int {
3   (fix f. fn k: Int {
4     if k == 0 then 1 else k * f(k - 1)
5   })(n)
6 }

```

Typing sketch: $\vdash \text{fact} : \text{Int} \rightarrow \text{Int}$. Evaluation follows E-APP1 and E-LET-style reductions with E-APP1 substituting into the fixed point.

20 Glossary

Core The minimal typed lambda calculus Yelra is based on.

Desugaring Translation from surface syntax to core terms.

Progress/Preservation The two pillars of type soundness.

Acknowledgments

We thank contributors who refined the lexer/parser and clarified edge cases in desugaring.